

Datenstruktur `ArrayList` in Java

Kriterium	Beschreibung	Wann zu verwenden?
Dynamische Größe	Eine <code>ArrayList</code> passt ihre Größe dynamisch an, wenn Elemente hinzugefügt oder entfernt werden.	Wenn die Anzahl der Elemente zur Laufzeit variabel ist und häufige Änderungen erforderlich sind.
Flexibilität bei der Datentypen	Eine <code>ArrayList</code> speichert Objekte und kann mit generischen Datentypen verwendet werden, z.B. <code>ArrayList<String></code> .	Wenn eine Sammlung von Objekten benötigt wird, deren Datentyp während der Programmaufzeit flexibel bleibt.
Automatische Erweiterung	Wenn mehr Platz benötigt wird, wird die <code>ArrayList</code> automatisch vergrößert.	Wenn die genaue Anzahl der benötigten Elemente nicht im Voraus bekannt ist oder dynamische Speicherverwaltung erforderlich ist.
Zugriffszeit (Indexzugriff)	Der Zugriff auf Elemente über den Index ist bei einer <code>ArrayList</code> im Allgemeinen $O(1)$, jedoch langsamer als bei einem Array.	Wenn Zugriff auf Elemente über ihren Index erforderlich ist, aber nicht die höchste Performanz nötig ist.
Verwaltung von <code>null</code>-Werten	Eine <code>ArrayList</code> kann <code>null</code> -Werte speichern, und diese sind in der Sammlung als gültige Objekte betrachtet.	Wenn <code>null</code> als gültiger Wert innerhalb der Sammlung gespeichert werden muss.
Wachsen bei Bedarf	Eine <code>ArrayList</code> wächst automatisch, wenn neue Elemente hinzugefügt werden.	Wenn es erforderlich ist, die Größe der Sammlung nach Bedarf zu erhöhen, ohne sich um die Verwaltung der Größe kümmern zu müssen.
Effizienz bei Einfügungen und Löschungen	Einfügen oder Löschen von Elementen am Ende der Liste ist effizient ($O(1)$), jedoch sind Einfügungen und Löschungen in der Mitte der Liste langsamer ($O(n)$).	Wenn viele Einfügungen oder Löschungen am Ende der Liste erwartet werden, aber bei Operationen in der Mitte oder am Anfang die Leistung keine Rolle spielt.
Speicherverwaltung	Eine <code>ArrayList</code> benötigt mehr Speicher als ein Array aufgrund der zusätzlichen Struktur und Verwaltung.	Wenn die Flexibilität und die automatische Größenanpassung der <code>ArrayList</code> die zusätzlichen Speicherkosten rechtfertigt.

Kriterium	Beschreibung	Wann zu verwenden?
Iteration und Streams	Eine <code>ArrayList</code> kann leicht mit einer <code>for</code> -Schleife oder Streams iteriert werden.	Wenn häufige Iterationen über alle Elemente erforderlich sind und das Arbeiten mit Streams und Lambdas erforderlich ist.
Weniger Kontrolle über Speicher	Im Vergleich zu einem Array hat die <code>ArrayList</code> weniger Kontrolle über den physischen Speicher, da sie intern Objektreferenzen verwendet.	Wenn keine präzise Kontrolle über den Speicher oder die Array-Implementierung erforderlich ist.
Synchronisierung	Eine <code>ArrayList</code> ist nicht threadsicher. Bei der Verwendung in einem multithreaded Kontext ist eine zusätzliche Synchronisierung erforderlich.	Wenn keine Synchronisierung in einem Multithreading-Kontext erforderlich ist oder Synchronisierung nachträglich hinzugefügt werden kann.

Zusammenfassung

Die Verwendung einer `ArrayList` ist besonders sinnvoll, wenn:

- Die Anzahl der Elemente während der Laufzeit variiert und häufig angepasst werden muss.
- Eine dynamische Größenanpassung gewünscht ist, ohne manuelle Speicherverwaltung.
- Häufige Iterationen über die Sammlung erforderlich sind.
- Der Zugriff auf die Elemente über den Index wichtig ist, ohne die allerhöchste Performanzanforderung.